

Better figures with ggplot2

Today's agenda:

- ggplot2 (R for Data Science Chapter 1)

ggplot2

- A package in the tidyverse
- “gg” refers to “grammar of graphics”, a consistent format for plotting
- Offers a lot of options for publication-quality figures
- Additional packages (usually starting with “gg”) add more features
- Syntax can be tricky at first

Getting set up

```
library(tidyverse)
```

```
library(ggribes)
```

```
avo <- read_rds("https://tinyurl.com/avonetbirddata")
```

Go ahead and run this

ggribes is new. Install it first if you need to

Understanding ggplot

Today, we're going to work through some examples

We'll start with single variables, then work up to more complicated figures

Distributions

Let's start with the single variable case and look at distributions

Base R:

```
hist(avo$Hand.Wing.Index)
```

ggplot:

```
avo |>  
  ggplot(mapping = aes(x = Hand.Wing.Index)) +  
  geom_histogram()
```

Try this out now and examine the output

Distributions

You could also use a density plot:

```
avo |>
```

```
  ggplot(mapping = aes(x = Hand.Wing.Index))+
```

```
  geom_density()
```

Try this out now and examine the output

ggplot syntax

```
avo |>
```

```
  ggplot(mapping = aes(x = Hand.Wing.Index)) +  
  geom_histogram()
```

Maps data to plot elements



Tells ggplot what type of plot to make



What happens if we omit the “geom_histogram” bit?

ggplot syntax

```
avo |>
```

```
  ggplot(mapping = aes(x = Hand.Wing.Index)) +  
  geom_histogram()
```



Plot functions begin with “geom_”

What are some of the other “geom_” options? (hint: use tab completion)

Do they result in a figure if we call them instead of geom_histogram?

Two variables

Options depend on type of data:

		y	
		Continuous	Categorical
x	Continuous	geom_point, geom_line	geom_density_ridges, coord_flip()-ed versions
	Categorical	geom_bar, geom_boxplot, geom_violin	geom_tile, geom_raster,

Continuous x, continuous y

geom_point

```
avo |>  
  ggplot(mapping = aes(x = Hand.Wing.Index,  
                        y = Kipps.Distance))+  
  geom_point()
```

Try this out. Then, plot two other continuous variables

geom_point

```
avo |>  
  ggplot(mapping = aes(x = Hand.Wing.Index,  
                        y = Kipps.Distance))+  
  geom_point()
```

How would we plot a line showing the relationship between the two?

lines

Three main approaches:

`geom_line()` adds a line connecting adjacent points

- Doesn't work in this case

`geom_abline()` allows you to add a line you specify

- Good option here if we fit a line first

`geom_smooth()` fits a line for you and add it

- Good option here

geom_line()

```
avo |>  
  ggplot(mapping = aes(x = Hand.Wing.Index,  
                        y = Kipps.Distance))+  
  geom_line()
```

Try this out. Is it useful?

geom_line()

```
avo |>
  mutate(rounded_hwi = round(Hand.Wing.Index)) |>
  group_by(rounded_hwi) |>
  summarise(mean_kipps = mean(Kipps.Distance)) |>
  ggplot(mapping = aes(x = rounded_hwi,
                        y = mean_kipps))+
  geom_line()
```

Try this one. Is this useful?

geom_abline()

With this, we can add a fitted line to a plot.

First, we need to decide on a line, or model one

```
kipps_v_hwi_model <- avo |>  
  lm(formula = Kipps.Distance ~ Hand.Wing.Index)
```

What data do we need from the model to plot the fitted line?

geom_abline()

```
ggplot(data = avo, mapping = aes(x = Hand.Wing.Index,  
                                  y = Kipps.Distance))+  
  geom_point()+  
  geom_abline(slope = kipps_v_hwi_model$coefficients[2],  
              intercept = kipps_v_hwi_model$coefficients[1],  
              color = "blue")
```

Run this. How does it look? Any changes you'd want to make?

geom_smooth()

Fits a line for you

- By default uses a loess fit
- In many cases, we want an lm instead

```
ggplot(data = avo, mapping = aes(x = Hand.Wing.Index,  
                                  y = Kipps.Distance))+  
  geom_point()+  
  geom_smooth()
```

Run this now. How does the fitted line look?

geom_smooth()

```
ggplot(data = avo,  
       mapping = aes(x = Hand.Wing.Index,  
                      y = Kipps.Distance)) +  
  geom_point() +  
  geom_smooth(method = "lm")
```

Run this. How does it compare with the plot you made using `geom_abline`?

Categorical x , continuous y

geom_bar

```
avo |>  
  group_by(Order1) |>  
  summarise(mean_mass = mean(Mass)) |>  
  slice_max(n = 5, order_by = mean_mass) |>  
  ggplot(mapping = aes(y = mean_mass, x = Order1)) +  
  geom_bar(stat = "identity")
```

Try this out. Then, try it for different variable(s)

geom_bar

- By default, geom_bar uses count data.
- We forced it to use the raw data we gave it (mean_mass)
- Let's try the default setting

geom_bar

```
avo |>
  filter(Order1 %in% c("Struthioniformes",
                      "Cathartiformes",
                      "Gaviiformes",
                      "Sphenisciformes",
                      "Ciconiiformes")) |>
  ggplot(mapping = aes(x= Order1)) +
  geom_bar()
```

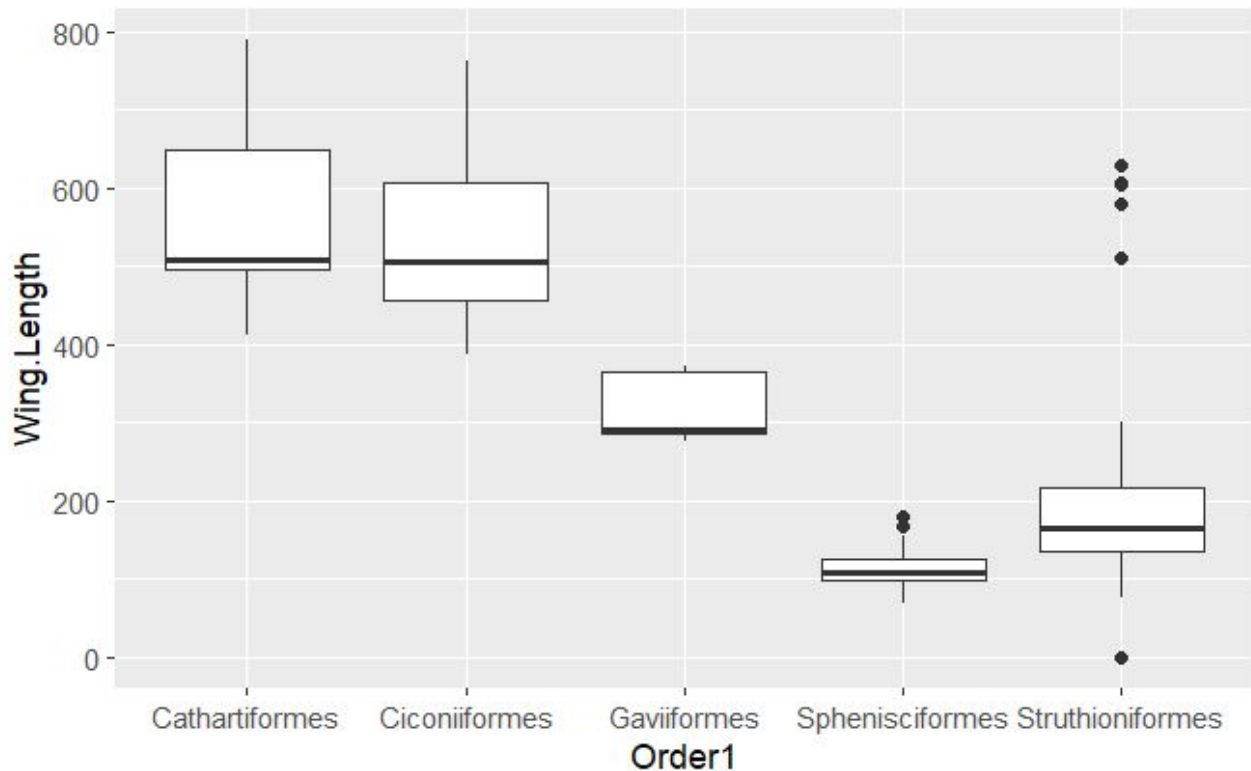
Try this one. Why don't we have to specify y in the aesthetics?

geom_boxplot

```
avo |>
  filter(Order1 %in% c("Struthioniformes",
                      "Cathartiformes",
                      "Gaviiformes",
                      "Sphenisciformes",
                      "Ciconiiformes")) |>
  ggplot(mapping = aes(x= Order1,
                      y = Wing.Length)) +
  geom_boxplot()
```

Run this now. Who remembers what the different features denote?

geom_boxplot



Horizontal line =
Median

Hinges = Q1 and Q3

Whiskers =
Furthest data within
 $1.5 \times \text{IQR}$ from Q1/Q3

$\text{IQR} = \text{Q3} - \text{Q1}$

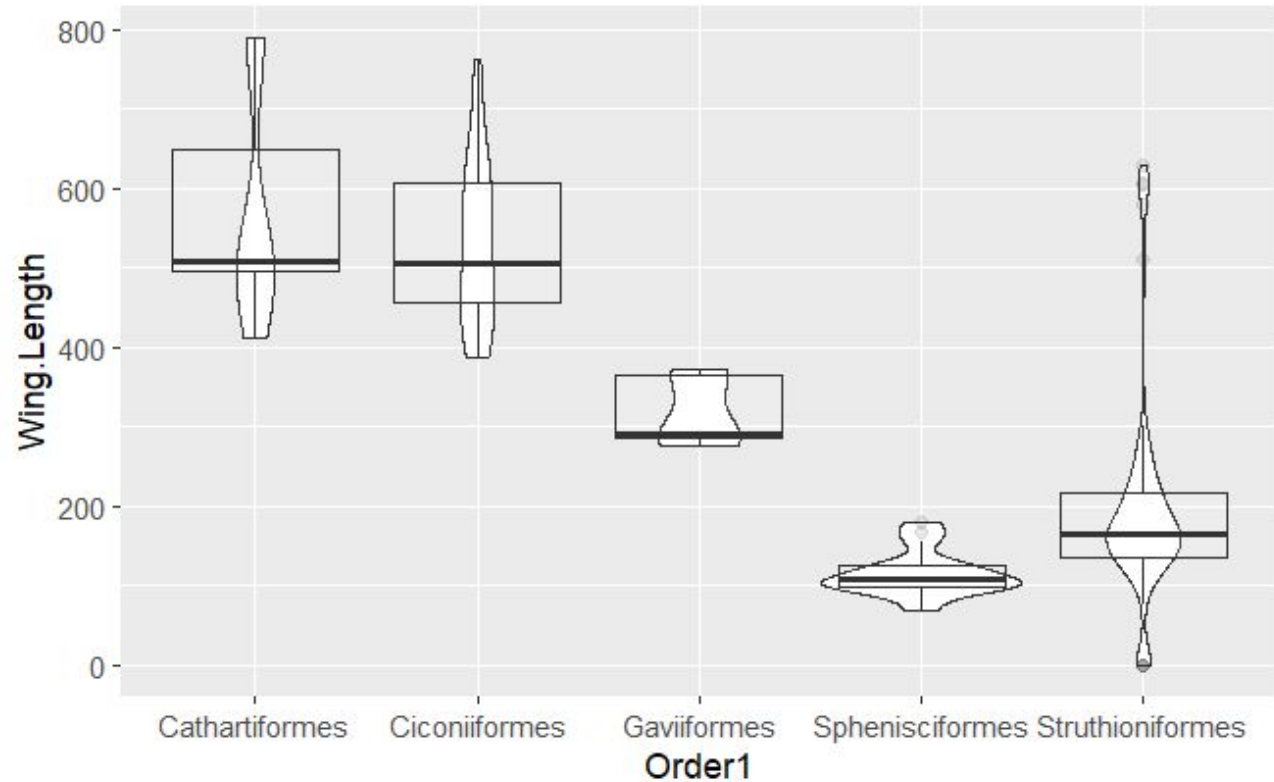
Dots = outliers (points
outside whiskers)

geom_violin

- A sort of combination of a density plot and a boxplot
- Contains more information than a boxplot

Revise the code we used for our boxplot to make a violin plot instead

Box vs violin



Continuous x, categorical y

Same functions as for categorical x and continuous y

- Just flip the x and y variables
- Can also use `coord_flip()`

```
avo |>
  group_by(Order1) |>
  summarise(mean_mass = mean(Mass)) |>
  slice_max(n = 5, order_by = mean_mass) |>
  ggplot(mapping = aes(y = mean_mass, x = Order1)) +
  geom_bar(stat = "identity")+
  coord_flip()
```

Try this one out

Joy Plots (aka Ridgeline plots)

- Inspired by Joy Division's album, *Unknown Pleasures* (1979)
- Essentially a stacked set of distributions
- Implemented in R using `geom_density_ridges()`



geom_density_ridges

```
avo |>
  group_by(Order1) |>
  mutate(mean_mass = mean(Mass)) |>
  filter(Order1 %in% c("Struthioniformes",
                      "Cathartiformes",
                      "Gaviiformes",
                      "Sphenisciformes",
                      "Ciconiiformes")) |>
  ggplot(mapping = aes(x = Wing.Length,
                      y = Order1,
                      group = Order1))+
  geom_density_ridges()
```

Try this out!

Categorical x, categorical y

Not a common thing

Sometimes done to show the counts in the different categories

geom_count

```
avo |>  
  filter(Family1 %in% c("Psittacidae", "Cacatuidae"))|>  
  ggplot(mapping = aes(x = Family1, y = Habitat))+  
  geom_count()
```

Try this out. Are there situations where it might be useful?

geom_tile

```
avo |>
  group_by(Family1, Trophic.Level) |>
  mutate(n_species = n(),
         log10_species = n() |> log10()) |>
  filter(Family1 %in% c("Psittacidae", "Cacatuidae")) |>
  ggplot(mapping = aes(x = Family1,
                       y = Trophic.Level,
                       fill = log10_species)) +
  geom_tile()
```

Try this one out as well.

More than 2 variables?

When you have more than two variables, options include:

- Using color/fill
- Using shape/line type
- Using size
- Using faceting

Color and fill

- Color used for lines and points, fill for polygons
- Example of using color to show three variables:

```
avo |>
  filter(Family1 %in% c("Psittacidae", "Cacatuidae")) |>
  group_by(Family1, Habitat) |>
  summarise(mean_mass = mean(Mass)) |>
  slice_max(n = 5, order_by = mean_mass) |>
  ggplot(mapping = aes(y = mean_mass,
                        x = Family1, fill = Habitat)) +
  geom_bar(stat = "identity", position = "dodge")
```

Try this one out

faceting

facet_wrap and facet_grid split up plots into different facets

Try out this example:

```
avo |>
  filter(Family1 %in% c("Psittacidae", "Cacatuidae"))|>
  ggplot(mapping = aes(x = Mass,
                        y = Wing.Length))+
  geom_point()+
  facet_wrap(~Family1)
```

faceting

Facet grid allows you to have plots divided into rows/columns

```
avo |>
  filter(Family1 %in% c("Psittacidae", "Cacatuidae"))|>
  ggplot(mapping = aes(x = Mass,
                        y = Wing.Length))+
  geom_point()+
  facet_grid(Family1 ~ Trophic.Niche)
```

Try this one out as well. Note the structure of the facet_grid arguments (y ~ x)

Labelling your figure

```
avo |>
  ggplot(mapping = aes(x = Hand.Wing.Index,
                        y = Kipps.Distance)) +
  geom_point(alpha = 0.1) +
  geom_smooth(method = "lm") +
  labs(title = "Kipp's distance rises with hand-wing index",
       x = "Hand-wing index",
       y = "Kipp's distance (mm)",
       caption = "Data: AVONET") +
  theme_bw()
```

One function, multiple labels.

Saving your figure

```
my_plot <- avo |>  
  ggplot(mapping = aes(x = Mass, y = Wing.Length)) +  
  geom_point()
```

```
ggsave(filename = "my_figure.png",  
        plot      = my_plot,  
        width     = 8,  
        height    = 5,  
        dpi       = 300)
```

No dev.off(). No blank Plots pane.

Leave out plot = and it saves the last one you made.

Soooo many more options

- What we've done today only scratches the surface
- Many ways of combining plots, customizing, etc.
- Lots of blogs and tutorials to look at once you have the basics down
- Check out the cheat sheet at:

<https://rstudio.github.io/cheatsheets/html/data-visualization.html>

Before next class:

- Read Bolker 3.1 to 3.3
- Next time: Deterministic functions
- Assignment 1 due tomorrow, Friday, 11:59 pm